Architectural Blueprint: Authentication & Authorization System (RBAC)

This document provides the technical specifications required to implement a secure Role-Based Access Control (RBAC) and Login system. It interfaces with the existing core HR database, utilizing a Python backend and a TypeScript frontend.

---

1. Database Layer (Security & RBAC Extensions)

To isolate public employee information from security credentials and permissions, we extend the relational model with three distinct tables.

Data Definition Language (DDL)

```
-- 1. System Roles Dictionary
CREATE TABLE ROLES (
    id SERIAL PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL UNIQUE -- 'Admin', 'HR', 'Employee'
);

-- 2. Security Credentials (1-to-1 Relationship with EMPLOYEES)
CREATE TABLE USER_CREDENTIALS (
    employee_id INT PRIMARY KEY REFERENCES EMPLOYEES(id) ON DELETE CASCADE,
    password_hash VARCHAR(255) NOT NULL, -- NEVER store plain-text passwords
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 3. Junction Table for Many-to-Many Assignment
CREATE TABLE EMPLOYEE_ROLES (
    employee_id INT REFERENCES EMPLOYEES(id) ON DELETE CASCADE,
    role_id INT REFERENCES ROLES(id) ON DELETE CASCADE,
    PRIMARY KEY (employee_id, role_id)
);
```

2. Backend API Layer (Python / FastAPI)

The backend authenticates users using **OAuth2 with JWT (JSON Web Tokens)**. The system relies on cryptographically signed tokens containing stateless user payload details, eliminating the need to query the database session state on every network request.

The Authentication Workflow

1. **Request:** The client sends credentials (`email` and `password`) via a JSON payload to the login route.
2. **Verification:** The backend fetches the `password_hash` linked to the email and verifies it securely using a modern hashing utility like `bcrypt` or `argon2`.
3. **Issuance:** On verification success, the server signs and returns a short-lived **JWT token** containing claims (`employee_id`, `roles`, `expiration_timestamp`).

Auth API Endpoints

Method	Endpoint	Description	Access Level
`POST`	`/api/v1/auth/login`	Validates credentials, issues JWT token access string	Public
`GET`	`/api/v1/auth/me`	Returns profile information of the current authenticated user	Valid JWT Token Required

Role-Based Route Protection (RBAC)

FastAPI dependencies act as gatekeepers on existing endpoints by decoding the JWT claims before executing data layer mutations:

- **Strict Restrictions:** Routes such as `POST /api/v1/employees` block execution unless the token metadata contains the `Admin` or `HR` role identifier.
- **Self-Service Verification:** When an `Employee` requests `/api/v1/employees/{id}/payslips`, the API middleware cross-references the targeted path `{id}` against the caller's encoded token `employee_id` to prevent vertical privilege escalations.

3. Frontend Dashboard Layer (TypeScript) — ✅ IMPLEMENTED

The client application captures the network state token to dynamically manage routing view visibility and enforce global type-safety contracts. **This layer has been fully implemented** using Next.js 16 App Router with React 19.

TypeScript Context Interface (`/types/auth.ts`) — ✅ Implemented

```
export interface AuthUser {
  id: number;
  name: string;
  email: string;
  roles: ('Admin' | 'HR' | 'Employee')[];
}

export interface AuthContextState {
  isAuthenticated: boolean;
  user: AuthUser | null;
  token: string | null;
  login: (token: string) => Promise<void>;
  logout: () => void;
}
```

Auth Context Provider (`/context/AuthContext.tsx`) — ✅ Implemented

The `AuthProvider` component manages the full JWT lifecycle:

1. **Initialization:** On mount, checks `localStorage` for a persisted `token`. If found, calls `/auth/me` to rehydrate the user session.
2. **Login:** Stores the JWT in `localStorage`, sets state, and fetches the user profile.
3. **Logout:** Clears `localStorage` and resets user/token state to `null`.
4. **Loading State:** Renders a "Loading session..." screen while rehydrating, preventing flash-of-unauthenticated-content.
5. **Custom Hook:** `useAuth()` provides type-safe access with a guard ensuring it's used within `AuthProvider`.

Route Guard Component (`/components/ProtectedRoute.tsx`) — ✅ Implemented

This wrapper component guards client views using Next.js `useRouter` for navigation:

```
'use client';

import React, { useEffect } from 'react';
import { useAuth } from '../context/AuthContext';
import { useRouter } from 'next/navigation';

interface ProtectedRouteProps {
  children: React.ReactNode;
  allowedRoles?: ('Admin' | 'HR' | 'Employee')[];
}

export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children,
allowedRoles }) => {
  const { isAuthenticated, user } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!isAuthenticated) {
      router.push('/login');
    }
  }, [isAuthenticated, router]);

  if (!isAuthenticated || !user) {
    return null;
  }

  if (allowedRoles && !user.roles.some((role) => allowedRoles.includes(role))) {
    return <div className="p-8 text-center text-red-600">Unauthorized. You do not
have permission to view this resource.</div>;
  }

  return <>{children}</>;
};
```

Implementation Notes

- **Login Page (`/src/app/login/page.tsx`):** Sends credentials as `FormData` (matching OAuth2's `application/x-www-form-urlencoded` expectations) to `POST /auth/login` and calls `login(access_token)` on success.
- **Dashboard Page (`/src/app/dashboard/page.tsx`):** Wrapped in `<ProtectedRoute>`, with sidebar navigation links conditionally rendered based on `user.roles` (e.g., "Employees" link only visible to `Admin` / `HR` roles).
- **Pending:** Dedicated `/unauthorized` error page not yet created — the `ProtectedRoute` currently renders an inline error message instead of redirecting.